



## Taller 2

### Objetivos

- Implementar algoritmos sobre árboles binarios de búsqueda (ABB/BST).
- Diseñar algoritmos basados en árboles binarios de búsqueda.

### Preámbulo

#### Entidades

Dentro de este taller, se trabajará con un ABB especial el cual sólo contendrá ceros y unos. Su regla especial es que cada camino dentro del mismo representará un número binario. Por ejemplo, si tenemos el siguiente árbol y tomamos el camino `root->right->right->right`, obtenemos el número binario 0111:

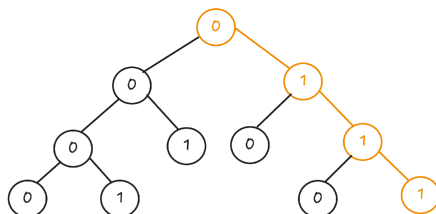


Figura 1: ABB con el camino ejemplificado resaltado en naranja

Cabe destacar que en este árbol los ceros siempre estarán como hijo izquierdo y los unos como hijo derecho.

En cuanto a la implementación de los `structs` que componen al ABB dentro del código, su definición es la siguiente (archivo: `src/libedd/libedd_bst.h`):

**libedd\_bst.h**

```
1 typedef struct bst_node {
2     int key;
3     struct bst_node *parent;
4     struct bst_node *left;
5     struct bst_node *right;
6 } BstNode;
7 typedef char (*MovementFunction)(BstNode *, int);
8 typedef struct bst {
9     BstNode *root;
10    size_t size;
11    MovementFunction move_to;
12 } Bst;
```

La `MovementFunction` no es importante, por lo que pueden ignorarla.

Dispondrás de las siguientes funciones que te serán de ayuda en el taller:

1. `BstNode *bst_node_create(int key)`: Función que crea un nuevo `BstNode` con su atributo `key` igual al argumento del mismo nombre, y retorna un puntero al nuevo nodo.
2. `int bst_node_destroy(EddError *err, BstNode *node)`: Función que elimina el nodo `node`.
3. `void bst_print(EddError *err, Bst *bst, FILE *output_file)`: Función que, dado un ABB `bst`, imprime sus contenidos en el archivo `output_file`. En caso de querer imprimir en consola, puede llamarse a la función con `output_file` igual a `NULL`.

## Eventos

### a) Sección Guiada: SEARCH-WORD

En esta parte deberás implementar la **función recursiva**:

```
1 | bool bst_search_word(BstNode *node, size_t length,  
2 | int *word, size_t idx)
```

Esta función recibe un nodo de un ABB, un arreglo de números **word** que representa cada dígito de un número binario, su largo y un índice. Su objetivo es ver si existe un camino en el ABB que represente al número binario **word**, retornando **true** si se encuentra y **false** en otro caso.

Para esto utilizaremos el siguiente algoritmo:

---

```
bst_search_word(BstNode *node, size_t length, int *word, size_t idx)
```

---

```
1: if idx = length then return true  
2: end if  
3: if node es nulo then return false  
4: end if  
5: if current->key ≠ word[idx] then return false  
6: end if  
7:  
8: next ← nulo  
9: new_idx ← idx  
10: if idx < (length - 1) then  
11:   new_idx ← new_idx + 1  
12: end if  
13:  
14: if word[new_idx] = 0 then  
15:   next ← hijo izquierdo de node  
16: else  
17:   next ← hijo derecho de node  
18: end if  
19: return bst_search_word(next, length, word, idx + 1)
```

---

### b) Sección Autónoma: WITH-PREFIX

En esta sección deberás, dado un número binario **prefix** contar la cantidad de números binarios que existan en el ABB que tengan a **prefix** como prefijo (incluyéndose a sí mismo).

Por ejemplo, si tenemos el árbol de ejemplo del comienzo, y usamos el prefijo 01, los números binarios que tiene el ABB que comienzan por este prefijo serían 5: 01, 010, 011, 0110, 0111.

Para esto, deberás implementar la función:

```
1 | int bst_with_prefix(Bst *bst, size_t prefix_size, int *prefix)
```

Donde **prefix\_size** es el tamaño del prefijo, y **prefix** es un arreglo de números con los dígitos del prefijo. Esta función debe retornar un **int** correspondiente a cantidad de números binarios encontrados en el ABB que empiezan con **prefix**.

## Input

Primero recibirás un entero **E** que indica la cantidad de eventos a procesar. Luego de ello, recibirás **E** eventos, donde cada uno representa un llamado a una de las funciones que debes implementar.

El formato de cada evento es el siguiente:

- **SEARCH-WORD:** Se recibe el nombre del evento seguido de un entero que representa el largo del número a buscar y una cadena de dígitos que representa el número binario en sí.
- **WITH-PREFIX:** Se recibe el nombre del evento seguido de un entero que representa el largo del prefijo y una cadena de dígitos que representa el prefijo en sí.

Como ejemplo tenemos el siguiente input:

| input |                    |
|-------|--------------------|
| 1     | 9                  |
| 2     | INSERT-WORD 3 010  |
| 3     | INSERT-WORD 3 000  |
| 4     | INSERT-WORD 3 001  |
| 5     | INSERT-WORD 3 011  |
| 6     | INSERT-WORD 4 0110 |
| 7     | INSERT-WORD 4 0111 |
| 8     | SEARCH-WORD 4 0001 |
| 9     | SEARCH-WORD 4 0110 |
| 10    | WITH-PREFIX 2 01   |

## Output

| output |                                                 |
|--------|-------------------------------------------------|
| 1      | Se inserto el numero binario 010                |
| 2      | Se inserto el numero binario 000                |
| 3      | Se inserto el numero binario 001                |
| 4      | Se inserto el numero binario 011                |
| 5      | Se inserto el numero binario 0110               |
| 6      | Se inserto el numero binario 0111               |
| 7      | No se encontro el numero binario 0001 en el ABB |
| 8      | Se encontro el numero binario 0110 en el ABB    |
| 9      | Cantidad de numeros binarios con prefijo 01: 5  |

## Entorno de trabajo

En su carpeta de trabajo tendrán al menos los siguientes elementos:

- `src/`: carpeta con su código
- `Makefile`: archivo para compilar
- `tests/`: carpetas de tests públicos

## Comandos (Terminal)

Para compilar:

```
make
```

Para ejecutar un test:

```
./PROGRAMA RUTA_ARCHIVO_INPUT RUTA_ARCHIVO_OUTPUT
```

Donde `PROGRAMA` es el nombre del archivo ejecutable que se genera al compilar, el cual puedes encontrar en la raíz de la carpeta de trabajo. Ejemplos de ejecutables: `troops`.

Además, `RUTA_ARCHIVO_INPUT` es la ruta al archivo de input del test y `RUTA_ARCHIVO_OUTPUT` es la ruta al archivo de output generado por el programa para el test. Ejemplo de `RUTA_ARCHIVO_INPUT` y `RUTA_ARCHIVO_OUTPUT`: `tests/easy-0/input.txt` y `output.txt`.

**Importante:** Para `RUTA_ARCHIVO_OUTPUT` no uses el archivo de output que viene dentro de un test. El parámetro `RUTA_ARCHIVO_OUTPUT` indica dónde tu programa escribirá el output que genere, por lo que usar una ruta como por ejemplo: `tests/test-1/output.txt` lo que producirá es que tu programa sobrescriba el output esperado por el test.

Si quieres verificar que el output generado por tu código es igual al esperado en un test (es decir, tu código pasa el test), puedes hacerlo con el siguiente comando:

```
diff RUTA_OUTPUT_GENERADO RUTA_OUTPUT_ESPERADO
```

donde `RUTA_OUTPUT_ESPERADO` será la ruta a un archivo con el output esperado del test, y `RUTA_OUTPUT_GENERADO` la ruta del archivo con el output de tu programa para dicho test. Si este comando finaliza sin imprimir nada en consola, significa que los archivos que entregaste son idénticos, y por tanto que el test pasa correctamente. En caso contrario, lo que se imprime es la diferencia entre los dos archivos, lo que podría ayudarte a ver porqué el test falló.

Para debuggear errores y/o leaks de memoria usando **valgrind**:

```
valgrind ./PROGRAMA RUTA_ARCHIVO_INPUT RUTA_ARCHIVO_OUTPUT
```

Para ejecutar todos los tests:

```
make test
```

## Proceso de Entrega

### Conexión a internet

Una vez se inicie la imagen y aparezca el escritorio, selecciona el ícono de redes abajo a la derecha. Luego: "Available Networks" → "eduroam". Para conectarse a eduroam, deben tener la siguiente configuración:

- En la opción de "Authentication" seleccionar "Tunneled TLS".
- Tener seleccionada la opción "No CA certificate is required".
- En la opción de "Inner authentication" seleccionar "PAP"
- Luego debe ingresar tus credenciales usuario UC. **IMPORTANTE:** en el usuario debes incluir el @, no sólo el nombre. Ejemplos correctos: "nombre@uc.cl", "nombre@estudiante.uc.cl". Incorrecto: 'nombre' sin el @.

### Descargar taller

Para descargar el código base que con el que trabajarán, deben ingresar al Canvas del curso usando el navegador (Firefox). Aquí encontrarán un archivo .zip con el código base del taller. Puedes descomprimir el archivo **usando el explorador de archivos**.

### Entregar Taller

Para entregar, simplemente deben comprimir su carpeta de trabajo en un archivo .zip, y subir este archivo comprimido al buzón de Canvas que tendrán disponible para el taller. Para comprimir su carpeta de trabajo pueden hacerlo **desde el explorador de archivos**.

## Evaluación

La nota de tu taller es calculada a partir de test cases privados de input/output que se evaluarán cada uno en forma binaria, es decir, 1 punto por output correcto y 0 puntos por output incorrecto.

Se entregarán test cases públicos para que puedas probar tu implementación durante el taller.

## Integridad académica e IA

Cualquier acto deshonesto o fraude académico está estrictamente prohibido. Se prohíbe el uso de herramientas de inteligencia artificial y el uso o manipulación de otros dispositivos electrónicos como laptops, tablets, y/o teléfonos durante la evaluación.